NEURAL NETWORKS – LESSON 1

24/05/2025

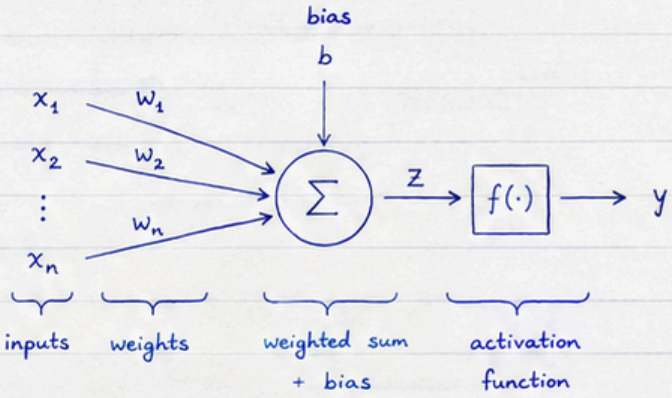
What Exactly Is a Neural Network?

1. The central idea

A neuron takes inputs $x_1, x_2, \dots, x_n$, multiplies each by a weight, adds them together, adds a bias, then applies a non-linear activation function f.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$
$$= w^T x + b \quad \text{(linear part)}$$

$$y = f(z) \quad \text{(activation)}$$



2. Inputs x_i

Example (material descriptors):

- x_1 = atomic number
- x_2 = electronegativity
- x_3 = atomic radius

One material:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 26 \\ 1.83 \\ 126 \end{bmatrix}$$

3. Weights w_i

$$W = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}, \quad \text{e.g. } w_1 = 0.2,\ w_2 = -0.5,\ w_3 = 0.1.$$

Weights determine how strongly each input affects the output. They are learned from data.

4. Bias b

- Adds flexibility (like the intercept in linear regression).
- Without b, the model must pass through the origin.

5. Example: one neuron (ReLU)

Let
$x_1 = 2,\ x_2 = 3$
$w_1 = 0.5,\ w_2 = -0.2$
$b = 0.1$

$$z = w_1 x_1 + w_2 x_2 + b$$
$$= (0.5)(2) + (-0.2)(3) + 0.1$$
$$= 1 - 0.6 + 0.1$$
$$\boxed{z = 0.5}$$

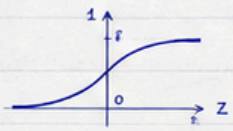
ReLU: $a = \max(0, z) = \boxed{0.5}$

If $z = -0.5$
$\Rightarrow a = \max(0, -0.5) = 0$

6. Activation functions

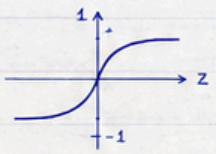
Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



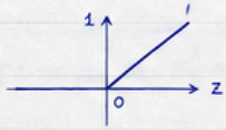
Tanh

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



ReLU

$$\text{ReLU}(z) = \max(0, z)$$



7. From one neuron to a layer

For 4 neurons and input $x \in \mathbb{R}^3$

$$z = Wx + b, \quad a = f(z)$$

where

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix}$$

8. From one layer to a network

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad a^{(1)} = f(z^{(1)}) \quad \rightarrow \quad z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad a^{(2)} = f(z^{(2)}) \quad \cdots \Rightarrow \hat{y} \text{ (output)}$$

input x → Layer 1 (hidden) → Layer 2 (hidden) → ⋯ → Output $\hat{y}$

9. Where is the learning?

- Parameters $W^{(l)}, b^{(l)}$ are initialized (e.g. randomly).
- Make prediction $\hat{y}$ for true target y.
- Compute loss $\mathcal{L}(y, \hat{y})$ (e.g. $\mathcal{L} = (y - \hat{y})^2$).
- Compute gradients $\frac{\partial \mathcal{L}}{\partial W^{(l)}}, \frac{\partial \mathcal{L}}{\partial b^{(l)}}$.
- Update parameters (gradient descent):

$$\theta_{new} = \theta_{old} - \eta \frac{\partial \mathcal{L}}{\partial \theta} \quad \text{(learning rate } \eta\text{)}$$

10. The complete training cycle

Input data → Forward pass → Prediction $\hat{y}$ → Loss $\mathcal{L}$ → Backprop (gradients) → Update parameters → (back to Forward pass)

11. First code: one neuron in NumPy

```
import numpy as np
x = np.array([2.0, 3.0])        # inputs
w = np.array([0.5, -0.2])       # weights
b = 0.1                         # bias
z = np.dot(w, x) + b            # linear part
a = max(0, z)                   # ReLU
print(z)    # 0.5
print(a)    # 0.5
```

12. Quick exercise (done!)

Given $x = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}, \quad W = \begin{bmatrix} 0.5 \\ -0.3 \\ 0.8 \end{bmatrix}, \quad b = 0.2$

$$z = W^T x + b = (0.5)(1) + (-0.3)(2) + (0.8)(-1) + 0.2 = \boxed{z = -0.7}$$

$$a = \text{ReLU}(z) = \max(0, -0.7) = 0 \quad \boxed{a = 0} \ \checkmark$$

Next: Lesson 2 – From a Single Neuron to a Layer (matrices, dimensions, NumPy implementation) ☺